PLAYWRIGHT AUTOMATION

WITH JAVA

Complete Notes: Basics to Advanced + API Testing + Reporting

SECTION 1 – INTRODUCTION TO PLAYWRIGHT

What is Playwright?

Playwright is an open-source test automation framework developed by Microsoft. It supports end-to-end testing of web applications across multiple browsers.

Developed by	Microsoft
Language support	Java, JavaScript/TypeScript, Python, C#
Browsers	Chromium, Firefox, WebKit (Safari)
Protocol	CDP (Chrome DevTools Protocol) + Browser-specific protocols
License	Apache 2.0 (Open Source)
Latest version	1.x (regularly updated)

Key Features

- Auto-waiting: Playwright automatically waits for elements to be actionable
- Network interception: Intercept, mock, or modify HTTP requests/responses
- Multiple contexts: Isolated browser contexts like incognito profiles
- Parallel execution: Run tests simultaneously across browsers
- Headless & headed mode: Run with or without browser UI
- Built-in assertions: Rich set of web-specific assertions
- Tracing: Record trace files for debugging
- Screenshots & Video: Capture on failure or always
- Mobile emulation: Emulate mobile devices and viewports
- API testing: Make HTTP requests directly from tests

Playwright vs Selenium

Feature	Playwright	Selenium
Auto-waiting	Yes (built-in)	No (manual waits)
Speed	Fast	Moderate
Network mock	Built-in	Needs extra libs
Multiple tabs	Easy	Complex
Flakiness	Low	Higher
Browser support	Chromium,Firefox,WebKit	All major browsers
Setup	Simple	Complex (drivers)

SECTION 2 – PROJECT SETUP & INSTALLATION

Prerequisites

Java: JDK 8 or above
Build Tool: Maven or Gradle
IDE: IntelliJ IDEA / Eclipse / VS Code

Maven pom.xml Dependencies

```
<dependencies>
  <!-- Playwright Java -->
  <dependency>
    <groupId>com.microsoft.playwright</groupId>
    <artifactId>playwright</artifactId>
    <version>1.44.0</version>
  </dependency>

  <!-- TestNG -->
  <dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>7.9.0</version>
  </dependency>

  <!-- Allure for Reporting -->
  <dependency>
    <groupId>io.qameta.allure</groupId>
    <artifactId>allure-testng</artifactId>
    <version>2.25.0</version>
  </dependency>

  <!-- Extent Reports -->
  <dependency>
    <groupId>com.aventstack</groupId>
    <artifactId>extentreports</artifactId>
    <version>5.1.1</version>
  </dependency>
</dependencies>
```

Install Playwright Browsers

After adding the dependency, run the following command to download browser binaries:

```
# Using Maven plugin
mvn exec:java -e -D exec.mainClass=com.microsoft.playwright.CLI -D exec.args="install"

# Or programmatically in code
// Playwright will auto-install browsers when first run
// Or use CLI: playwright install
```

Project Structure

```
playwright-java-project/
├── src/
│   ├── main/java/
│   │   ├── pages/          ← Page Object classes
│   │   ├── utils/          ← Helper utilities
│   │   └── base/           ← Base classes
│   └── test/java/
│       └── tests/          ← Test classes
├── resources/
│   ├── config.properties   ← Config data
│   └── testng.xml          ← TestNG suite
├── reports/                ← Generated reports
└── pom.xml
```

SECTION 3 – CORE CONCEPTS & BASIC USAGE

Playwright Object Hierarchy

```
Playwright                    ← Entry point
  └── Browser                 ← Browser instance (Chromium/Firefox/WebKit)
        └── BrowserContext    ← Isolated session (like incognito)
              └── Page        ← Single tab / window
                    └── Frame ← iframe or main frame
```

Basic Test Structure

```
import com.microsoft.playwright.*;
import org.testng.annotations.*;

public class MyFirstTest {
    Playwright playwright;
    Browser browser;
    Page page;

    @BeforeClass
    public void setup() {
        playwright = Playwright.create();
        browser = playwright.chromium().launch(
            new BrowserType.LaunchOptions().setHeadless(false));
        page = browser.newPage();
    }

    @Test
    public void testTitle() {
        page.navigate("https://example.com");
        System.out.println(page.title());
        page.screenshot(new
Page.ScreenshotOptions().setPath(Paths.get("screenshot.png")));
    }

    @AfterClass
    public void teardown() {
        browser.close();
        playwright.close();
    }
}
```

Browser Launch Options

```
// Headless mode (default: true)
browser = playwright.chromium().launch(
    new BrowserType.LaunchOptions()
        .setHeadless(true)            // no UI
        .setSlowMo(500)               // slow down by 500ms
        .setArgs(List.of("--start-maximized"))
        .setTimeout(30000)            // launch timeout
);

// Browser Context options
BrowserContext context = browser.newContext(
```

```
    new Browser.NewContextOptions()
        .setViewportSize(1920, 1080)
        .setLocale("en-US")
        .setTimezoneId("Asia/Kolkata")
        .setIgnoreHTTPSErrors(true)   // ignore SSL errors
        .setRecordVideoDir(Paths.get("videos/"))  // record video
);
```

Locator Strategies

> 💡 Locator is the main way to find elements. Playwright recommends using role/text locators over CSS/XPath.

```
// ── CSS Selector ──────────────────────────────────────
page.locator("#username")                // by ID
page.locator(".btn-primary")             // by class
page.locator("input[name='email']")      // by attribute
page.locator("div > p:first-child")      // child combinator

// ── XPath ─────────────────────────────────────────────
page.locator("//input[@id='user']")      // XPath
page.locator("xpath=//button[text()='Login']")

// ── Text ──────────────────────────────────────────────
page.locator("text=Submit")              // exact text
page.getByText("Submit")                 // preferred
page.getByText("Submit", new Page.GetByTextOptions().setExact(true))

// ── Role (ARIA) ───────────────────────────────────────
page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Login"))
page.getByRole(AriaRole.TEXTBOX, new Page.GetByRoleOptions().setName("Email"))
page.getByRole(AriaRole.CHECKBOX)
page.getByRole(AriaRole.LINK, new Page.GetByRoleOptions().setName("Home"))

// ── Label / Placeholder / AltText ─────────────────────
page.getByLabel("Email Address")
page.getByPlaceholder("Enter email")
page.getByAltText("Profile Picture")
page.getByTitle("Close Dialog")

// ── Test ID ───────────────────────────────────────────
page.getByTestId("submit-btn")           // data-testid attribute

// ── Filter & Chain ────────────────────────────────────
page.locator(".list-item").filter(new Locator.FilterOptions().setHasText("Active"))
page.locator("ul").locator("li")         // chained
page.locator("li").nth(2)                // 0-based index
page.locator("li").first()
page.locator("li").last()
```

SECTION 4 – ACTIONS & INTERACTIONS

Click & Keyboard Actions

```
// Click actions
page.locator("#btn").click();
page.locator("#btn").dblclick();
page.locator("#btn").click(new Locator.ClickOptions()
    .setButton(MouseButton.RIGHT)     // right-click
    .setModifiers(List.of(KeyboardModifier.SHIFT)) // shift+click
    .setDelay(100));                  // click with delay

// Type / Fill
page.locator("#email").fill("user@test.com");  // clear & type
page.locator("#email").type("user@test.com");  // char by char
page.locator("#email").pressSequentially("hello", new
Locator.PressSequentiallyOptions().setDelay(100));

// Keyboard
page.keyboard().press("Enter");
page.keyboard().press("Control+A");
page.keyboard().type("Hello World");
page.locator("#search").press("Enter");

// Hover & Focus
page.locator("#menu").hover();
page.locator("#input").focus();
page.locator("#input").blur();

// Clear
page.locator("#field").clear();
page.locator("#field").fill("");
```

Dropdowns & Checkboxes

```
// Select dropdown by value/label/index
page.locator("#country").selectOption("IN");                    // by value
page.locator("#country").selectOption(new SelectOption().setLabel("India")); // by label
page.locator("#country").selectOption(new SelectOption().setIndex(2));       // by index
// Multi-select
page.locator("#multi").selectOption(new String[]{"opt1","opt2"});

// Checkbox
page.locator("#agree").check();          // check
page.locator("#agree").uncheck();        // uncheck
page.locator("#agree").setChecked(true); // set state
boolean checked = page.locator("#agree").isChecked();

// Radio button
page.locator("input[value='male']").check();
```

File Upload & Download

```
// File Upload
page.locator("input[type='file']").setInputFiles(Paths.get("upload/file.pdf"));
// Multiple files
```

```
page.locator("#upload").setInputFiles(new Path[]{
    Paths.get("file1.pdf"), Paths.get("file2.pdf")
});
// Remove file
page.locator("#upload").setInputFiles(new Path[0]);

// File Download
Download download = page.waitForDownload(() -> {
    page.locator("#download-btn").click();
});
download.saveAs(Paths.get("downloads/" + download.suggestedFilename()));
System.out.println("Downloaded: " + download.suggestedFilename());
```

Drag & Drop / Mouse

```
// Drag & Drop
page.locator("#source").dragTo(page.locator("#target"));

// Manual drag with mouse
page.mouse().move(100, 100);
page.mouse().down();
page.mouse().move(300, 300);
page.mouse().up();

// Scroll
page.mouse().wheel(0, 500);           // scroll down 500px
page.locator("#container").evaluate("el => el.scrollTop = 500");
```

SECTION 5 – WAITS & ASSERTIONS

Playwright Auto-Waiting

Playwright automatically waits for elements to be:

- Attached to the DOM
- Visible on screen
- Stable (not animating)
- Enabled (not disabled)
- Editable (for input actions)
- Receives events (not obscured by another element)

> 💡 You rarely need explicit waits. Playwright handles most waiting automatically!

Explicit Waits

```
// Wait for element state
page.locator("#loading").waitFor(new Locator.WaitForOptions()
    .setState(WaitForSelectorState.VISIBLE)  // VISIBLE, HIDDEN, ATTACHED, DETACHED
    .setTimeout(5000));

// Wait for navigation
page.waitForNavigation(() -> {
    page.locator("#login").click();
});

// Wait for URL
page.waitForURL("**/dashboard");
page.waitForURL(Pattern.compile(".*/dashboard.*"));

// Wait for load state
page.waitForLoadState(LoadState.NETWORKIDLE);
page.waitForLoadState(LoadState.DOMCONTENTLOADED);

// Wait for response
Response response = page.waitForResponse("**/api/users", () -> {
    page.locator("#load").click();
});

// Custom condition
page.waitForFunction("() => document.readyState === 'complete'");

// Fixed delay (avoid if possible)
page.waitForTimeout(2000);
```

Assertions (Playwright Assertions)

> 💡 Use PlaywrightAssertions for built-in web-aware assertions that auto-retry.

```
import com.microsoft.playwright.assertions.PlaywrightAssertions;
```

```
import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

// Page assertions
assertThat(page).hasTitle("Home Page");
assertThat(page).hasTitle(Pattern.compile(".*Home.*"));
assertThat(page).hasURL("https://example.com/home");
assertThat(page).hasURL(Pattern.compile(".*/home$"));

// Locator assertions
assertThat(page.locator("#msg")).isVisible();
assertThat(page.locator("#msg")).isHidden();
assertThat(page.locator("#btn")).isEnabled();
assertThat(page.locator("#btn")).isDisabled();
assertThat(page.locator("#chk")).isChecked();
assertThat(page.locator("h1")).hasText("Welcome");
assertThat(page.locator("h1")).hasText(Pattern.compile("Wel.*"));
assertThat(page.locator("#count")).containsText("5");
assertThat(page.locator("#input")).hasValue("hello");
assertThat(page.locator("#input")).isEmpty();
assertThat(page.locator(".items")).hasCount(5);
assertThat(page.locator("#box")).hasAttribute("class", "active");
assertThat(page.locator("#box")).hasCSS("color", "rgb(0, 0, 0)");

// Custom timeout for assertion
assertThat(page.locator("#spinner")).isHidden(
    new LocatorAssertions.IsHiddenOptions().setTimeout(10000));
```

SECTION 6 – PAGE OBJECT MODEL (POM)

What is POM?

Page Object Model is a design pattern where each web page or component is represented as a Java class. It separates test logic from page interactions, making tests more maintainable and readable.

Base Page Class

```
// base/BasePage.java
public class BasePage {
    protected Page page;

    public BasePage(Page page) {
        this.page = page;
    }

    public void navigateTo(String url) {
        page.navigate(url);
    }

    public String getTitle() {
        return page.title();
    }

    public void waitForElement(String selector) {
        page.locator(selector).waitFor();
    }
}
```

Login Page Object

```
// pages/LoginPage.java
public class LoginPage extends BasePage {

    // Locators as fields
    private final Locator usernameField;
    private final Locator passwordField;
    private final Locator loginButton;
    private final Locator errorMessage;

    public LoginPage(Page page) {
        super(page);
        usernameField = page.getByLabel("Username");
        passwordField = page.getByLabel("Password");
        loginButton   = page.getByRole(AriaRole.BUTTON, new
Page.GetByRoleOptions().setName("Login"));
        errorMessage  = page.locator(".error-message");
    }

    public void enterUsername(String username) {
        usernameField.fill(username);
    }

    public void enterPassword(String password) {
```

```
        passwordField.fill(password);
    }

    public DashboardPage clickLogin() {
        loginButton.click();
        return new DashboardPage(page);  // return next page
    }

    public LoginPage login(String user, String pass) {  // fluent
        enterUsername(user);
        enterPassword(pass);
        loginButton.click();
        return this;
    }

    public String getErrorMessage() {
        return errorMessage.textContent();
    }

    public boolean isErrorDisplayed() {
        return errorMessage.isVisible();
    }
}
```

Test Using POM

```
// tests/LoginTest.java
public class LoginTest extends BaseTest {

    @Test
    public void validLoginTest() {
        LoginPage loginPage = new LoginPage(page);
        loginPage.navigateTo("https://myapp.com/login");
        DashboardPage dashboard = loginPage
            .enterUsername("admin")
            // fluent style — or just:
            .clickLogin();

        assertThat(page).hasURL(Pattern.compile(".*/dashboard"));
    }

    @Test
    public void invalidLoginShowsError() {
        LoginPage loginPage = new LoginPage(page);
        loginPage.navigateTo("https://myapp.com/login");
        loginPage.enterUsername("wrong");
        loginPage.enterPassword("wrong");
        loginPage.clickLogin();

        assertThat(loginPage.getErrorLocator()).isVisible();
        assertEquals(loginPage.getErrorMessage(), "Invalid credentials");
    }
}
```

SECTION 7 – ADVANCED FEATURES

Handling Multiple Windows / Tabs

```
// Wait for new page/tab to open
Page newPage = context.waitForPage(() -> {
    page.locator("#open-new-tab").click();
});
newPage.waitForLoadState();
System.out.println("New tab URL: " + newPage.url());
newPage.close();

// Manually open new tab
Page tab2 = context.newPage();
tab2.navigate("https://another-site.com");

// Get all pages in context
List<Page> pages = context.pages();
```

iFrames

```
// Access element inside iframe
FrameLocator iframe = page.frameLocator("#my-iframe");
iframe.locator("#submit").click();
iframe.getByLabel("Name").fill("John");

// Nested iframes
page.frameLocator("#outer").frameLocator("#inner").locator("button").click();

// Get frame by name
Frame frame = page.frame("myFrame");
frame.locator("#input").fill("value");
```

Dialogs / Alerts

```
// Handle alert/confirm/prompt
page.onDialog(dialog -> {
    System.out.println("Dialog message: " + dialog.message());
    dialog.accept();             // click OK
    // dialog.dismiss();         // click Cancel
    // dialog.accept("text");    // type in prompt and accept
});
page.locator("#alert-btn").click();
```

Network Interception & Mocking

```
// Abort specific requests
page.route("**/*.png", route -> route.abort());

// Mock API response
page.route("**/api/users", route -> route.fulfill(new Route.FulfillOptions()
    .setStatus(200)
    .setContentType("application/json")
```

```
    .setBody("[{\"id\":1,\"name\":\"Mock User\"}]")
));

// Modify request headers
page.route("**/*", route -> route.resume(
    new Route.ResumeOptions().setHeaders(Map.of("X-Token","test123"))
));

// Intercept and log all requests
page.onRequest(request ->
    System.out.println(">> " + request.method() + " " + request.url())
);
page.onResponse(response ->
    System.out.println("<< " + response.status() + " " + response.url())
);
```

JavaScript Execution

```
// Execute JavaScript
Object result = page.evaluate("document.title");
Object count  = page.evaluate("document.querySelectorAll('li').length");

// Pass argument to JS
page.evaluate("selector => document.querySelector(selector).click()", "#btn");

// Scroll into view
page.locator("#footer").evaluate("el => el.scrollIntoView()");

// Inject script
page.addInitScript("window.isAutomated = true;");

// Get inner HTML
String html = page.locator("#container").innerHTML();
String text = page.locator("#msg").innerText();
String attr = page.locator("#link").getAttribute("href");
```

Authentication & Storage State

```
// Save login state (cookies + local storage)
context.storageState(new BrowserContext.StorageStateOptions()
    .setPath(Paths.get("auth/state.json")));

// Re-use saved state (skip login for every test)
BrowserContext context = browser.newContext(
    new Browser.NewContextOptions()
        .setStorageStatePath(Paths.get("auth/state.json"))
);

// Set cookies manually
context.addCookies(List.of(new Cookie("session","abc123")
    .setDomain(".example.com")
    .setPath("/")
    .setSecure(true)));

// Get cookies
List<Cookie> cookies = context.cookies();
context.clearCookies();
```

Screenshots & Video

```
// Full page screenshot
page.screenshot(new Page.ScreenshotOptions()
    .setPath(Paths.get("screenshots/full.png"))
    .setFullPage(true));

// Element screenshot
page.locator("#chart").screenshot(
    new Locator.ScreenshotOptions().setPath(Paths.get("chart.png")));

// Screenshot as byte array (for report attachment)
byte[] bytes = page.screenshot();

// Video recording — enable in context
BrowserContext context = browser.newContext(
    new Browser.NewContextOptions()
        .setRecordVideoDir(Paths.get("videos/"))
        .setRecordVideoSize(1280, 720)
);
Page page = context.newPage();
// ... run test ...
context.close();  // video saved on context close
System.out.println("Video: " + page.video().path());
```

Tracing

```
// Start tracing
context.tracing().start(new Tracing.StartOptions()
    .setScreenshots(true)
    .setSnapshots(true)
    .setSources(true));

// ... run test ...

// Stop and save trace
context.tracing().stop(new Tracing.StopOptions()
    .setPath(Paths.get("trace.zip")));

// View trace: npx playwright show-trace trace.zip
```

SECTION 8 – API TESTING WITH PLAYWRIGHT

Why Use Playwright for API Testing?

- Combine UI and API tests in the same framework
- Set up data via API before UI tests run
- Verify backend state after UI actions
- Faster execution than UI for data validation
- Reuse browser auth session in API calls

APIRequestContext Setup

```
import com.microsoft.playwright.*;

public class APITest {
    static Playwright playwright;
    static APIRequestContext apiContext;

    @BeforeClass
    public static void setup() {
        playwright = Playwright.create();
        apiContext = playwright.request().newContext(
            new APIRequest.NewContextOptions()
                .setBaseURL("https://reqres.in")
                .addHttpHeaders(Map.of(
                    "Accept",       "application/json",
                    "Content-Type", "application/json"
                ))
                .setTimeout(15000)
        );
    }

    @AfterClass
    public static void teardown() {
        apiContext.dispose();
        playwright.close();
    }
}
```

GET Request

```
// Simple GET
APIResponse response = apiContext.get("/api/users");
System.out.println("Status: " + response.status());  // 200
System.out.println("Body:   " + response.text());

// GET with query parameters
APIResponse response = apiContext.get("/api/users",
    RequestOptions.create().setQueryParam("page", 2));

// Parse JSON response with Jackson / Gson
String body = response.text();
ObjectMapper mapper = new ObjectMapper();
JsonNode json = mapper.readTree(body);
```

```
System.out.println(json.get("data").get(0).get("first_name").asText());

// Assert response
assertEquals(response.status(), 200);
assertTrue(response.ok());           // status 2xx
assertThat(response).isOK();         // Playwright assertion
```

POST Request

```
// POST with JSON body
APIResponse response = apiContext.post("/api/users",
    RequestOptions.create()
        .setData(Map.of(
            "name",  "John Doe",
            "job",   "QA Engineer"
        ))
);

assertEquals(response.status(), 201);
String body = response.text();
assertTrue(body.contains("John Doe"));

// POST with raw JSON string
String payload = "{\"name\":\"Jane\",\"job\":\"Dev\"}";
APIResponse res = apiContext.post("/api/users",
    RequestOptions.create().setData(payload)
);
```

PUT / PATCH / DELETE

```
// PUT — full update
APIResponse putRes = apiContext.put("/api/users/2",
    RequestOptions.create().setData(Map.of(
        "name", "Updated Name",
        "job",  "Senior QA"
    ))
);
assertEquals(putRes.status(), 200);

// PATCH — partial update
APIResponse patchRes = apiContext.patch("/api/users/2",
    RequestOptions.create().setData(Map.of("job", "Lead QA"))
);
assertEquals(patchRes.status(), 200);

// DELETE
APIResponse deleteRes = apiContext.delete("/api/users/2");
assertEquals(deleteRes.status(), 204);
```

Authentication in API Requests

```
// Bearer token
APIResponse res = apiContext.get("/api/protected",
    RequestOptions.create()
        .setHeader("Authorization", "Bearer " + token)
);

// Basic auth in context
```

```
apiContext = playwright.request().newContext(
    new APIRequest.NewContextOptions()
        .setBaseURL("https://api.example.com")
        .setHttpCredentials("username", "password")
);

// API key header
APIResponse res = apiContext.get("/data",
    RequestOptions.create()
        .setHeader("x-api-key", "your-api-key-here")
);
```

API + UI Combined Test

```
// Create user via API, verify in UI
@Test
public void createUserViaAPIAndVerifyInUI() {
    // Step 1: Create user via API
    APIResponse createRes = apiContext.post("/api/users",
        RequestOptions.create().setData(Map.of(
            "name", "TestUser", "email", "test@test.com",
            "password", "Pass@123"
        ))
    );
    assertEquals(createRes.status(), 201);
    String userId = new ObjectMapper().readTree(createRes.text()).get("id").asText();

    // Step 2: Log in via UI and verify user appears
    page.navigate("/admin/users");
    assertThat(page.locator("[data-id='" + userId + "']")).isVisible();

    // Step 3: Delete via API (cleanup)
    apiContext.delete("/api/users/" + userId);
}
```

SECTION 9 – BASE TEST CLASS & CONFIGURATION

BaseTest Class

```
// base/BaseTest.java
public class BaseTest {
    protected Playwright playwright;
    protected Browser browser;
    protected BrowserContext context;
    protected Page page;

    @BeforeMethod
    public void setUp(Method method) {
        playwright = Playwright.create();
        String browserName = System.getProperty("browser", "chromium");

        BrowserType.LaunchOptions launchOptions = new BrowserType.LaunchOptions()
            .setHeadless(Boolean.parseBoolean(System.getProperty("headless","true")));

        browser = switch (browserName) {
            case "firefox" -> playwright.firefox().launch(launchOptions);
            case "webkit"  -> playwright.webkit().launch(launchOptions);
            default        -> playwright.chromium().launch(launchOptions);
        };

        context = browser.newContext(new Browser.NewContextOptions()
            .setViewportSize(1920, 1080)
            .setIgnoreHTTPSErrors(true)
            .setRecordVideoDir(Paths.get("videos/"))
        );
        context.tracing().start(new Tracing.StartOptions()
            .setScreenshots(true).setSnapshots(true));
        page = context.newPage();
    }

    @AfterMethod
    public void tearDown(ITestResult result) {
        if (result.getStatus() == ITestResult.FAILURE) {
            // Capture screenshot on failure
            byte[] screenshot = page.screenshot();
            // Attach to report (see Reporting section)
        }
        context.tracing().stop(new Tracing.StopOptions()
            .setPath(Paths.get("traces/" + result.getName() + ".zip")));
        context.close();
        browser.close();
        playwright.close();
    }
}
```

config.properties

```
# config/config.properties
base.url=https://myapp.com
browser=chromium
headless=true
timeout=30000
api.base.url=https://api.myapp.com
```

```
api.token=your-token-here
```

ConfigReader Utility

```
// utils/ConfigReader.java
public class ConfigReader {
    private static Properties props = new Properties();

    static {
        try (InputStream is = ConfigReader.class
                .getResourceAsStream("/config.properties")) {
            props.load(is);
        } catch (Exception e) { throw new RuntimeException(e); }
    }

    public static String get(String key) { return props.getProperty(key); }
    public static int    getInt(String key) { return Integer.parseInt(get(key)); }
    public static boolean getBool(String key) { return Boolean.parseBoolean(get(key)); }
}

// Usage
page.navigate(ConfigReader.get("base.url"));
```

SECTION 10 – TEST REPORTING

Option 1: Allure Report (Recommended)

Allure generates beautiful interactive HTML reports with steps, screenshots, and timelines.

▸ Maven pom.xml — Allure Plugin

```
<build>
  <plugins>
    <plugin>
      <groupId>io.qameta.allure</groupId>
      <artifactId>allure-maven</artifactId>
      <version>2.12.0</version>
    </plugin>
  </plugins>
</build>
```

▸ Allure Annotations in Tests

```
import io.qameta.allure.*;

@Epic("User Management")
@Feature("Login")
public class LoginTest extends BaseTest {

    @Test
    @Story("Valid login redirects to dashboard")
    @Description("Tests that a valid user can log in successfully")
    @Severity(SeverityLevel.CRITICAL)
    public void validLoginTest() {
        Allure.step("Navigate to login page", () ->
            page.navigate(ConfigReader.get("base.url") + "/login")
        );
        Allure.step("Enter credentials", () -> {
            page.getByLabel("Username").fill("admin");
            page.getByLabel("Password").fill("admin123");
        });
        Allure.step("Click Login button", () ->
            page.getByRole(AriaRole.BUTTON, new
Page.GetByRoleOptions().setName("Login")).click()
        );
        Allure.step("Verify dashboard is shown", () ->
            assertThat(page).hasURL(Pattern.compile(".*/dashboard"))
        );
    }

    // Attach screenshot to Allure report
    @Attachment(value = "Screenshot", type = "image/png")
    public byte[] attachScreenshot() {
        return page.screenshot();
    }
}
```

▸ Generate Allure Report

```
# Run tests
mvn clean test
```

```
# Generate report
mvn allure:report

# Serve report in browser
mvn allure:serve
```

Option 2: ExtentReports

```
// utils/ExtentReportManager.java
public class ExtentReportManager {
    private static ExtentReports extent;
    private static ThreadLocal<ExtentTest> test = new ThreadLocal<>();

    public static ExtentReports getInstance() {
        if (extent == null) {
            ExtentSparkReporter spark = new
ExtentSparkReporter("reports/ExtentReport.html");
            spark.config().setDocumentTitle("Playwright Test Report");
            spark.config().setReportName("Automation Results");
            spark.config().setTheme(Theme.DARK);
            extent = new ExtentReports();
            extent.attachReporter(spark);
            extent.setSystemInfo("OS", System.getProperty("os.name"));
            extent.setSystemInfo("Java", System.getProperty("java.version"));
        }
        return extent;
    }

    public static ExtentTest getTest()              { return test.get(); }
    public static void setTest(ExtentTest extTest)  { test.set(extTest); }
}
```

Extent Report Listener (TestNG)

```
// listeners/ExtentReportListener.java
public class ExtentReportListener implements ITestListener {

    @Override
    public void onTestStart(ITestResult result) {
        ExtentTest test = ExtentReportManager.getInstance()
            .createTest(result.getMethod().getMethodName());
        ExtentReportManager.setTest(test);
    }

    @Override
    public void onTestSuccess(ITestResult result) {
        ExtentReportManager.getTest().pass("Test Passed");
    }

    @Override
    public void onTestFailure(ITestResult result) {
        ExtentReportManager.getTest().fail(result.getThrowable());
        // Attach screenshot
        // (get page from thread-local in your BaseTest)
        ExtentReportManager.getTest()
            .addScreenCaptureFromBase64String(getBase64Screenshot(), "Failure
Screenshot");
    }

    @Override
```

```
    public void onFinish(ITestContext context) {
        ExtentReportManager.getInstance().flush();
    }
}
```

- **testng.xml with Listener**

```
<suite name="PlaywrightSuite" parallel="methods" thread-count="4">
  <listeners>
    <listener class-name="listeners.ExtentReportListener"/>
  </listeners>
  <test name="Regression">
    <classes>
      <class name="tests.LoginTest"/>
      <class name="tests.DashboardTest"/>
    </classes>
  </test>
</suite>
```

SECTION 11 – PARALLEL & CROSS-BROWSER EXECUTION

Parallel Tests with TestNG

> 💡 Each thread must have its own Playwright/Browser/Page instance. Use @BeforeMethod (not @BeforeClass) for thread-safety.

```
// testng.xml — parallel at method level
<suite name="Parallel" parallel="methods" thread-count="5">
  <test name="ParallelTest">
    <classes>
      <class name="tests.SearchTest"/>
    </classes>
  </test>
</suite>

// Each @BeforeMethod creates fresh browser instance
// (already shown in BaseTest above — this is the correct pattern)
```

Cross-Browser TestNG DataProvider

```
@DataProvider(name = "browsers", parallel = true)
public Object[][] browsers() {
    return new Object[][] {{"chromium"}, {"firefox"}, {"webkit"}};
}

@Test(dataProvider = "browsers")
public void crossBrowserTest(String browserName) throws Exception {
    BrowserType.LaunchOptions opts = new BrowserType.LaunchOptions().setHeadless(true);
    Browser b = switch (browserName) {
        case "firefox" -> playwright.firefox().launch(opts);
        case "webkit"  -> playwright.webkit().launch(opts);
        default        -> playwright.chromium().launch(opts);
    };
    Page p = b.newPage();
    p.navigate("https://example.com");
    assertEquals(p.title(), "Example Domain");
    b.close();
}
```

Mobile Emulation

```
// Emulate iPhone 13
BrowserContext mobileContext = browser.newContext(
    playwright.devices().get("iPhone 13")  // built-in device descriptors
);

// Custom device
BrowserContext mobileContext = browser.newContext(
    new Browser.NewContextOptions()
        .setUserAgent("Mozilla/5.0 (iPhone; CPU iPhone OS 14_0...")
        .setViewportSize(390, 844)
        .setDeviceScaleFactor(3)
        .setIsMobile(true)
        .setHasTouch(true)
```

```
);
```

SECTION 12 – DATA-DRIVEN TESTING

TestNG DataProvider

```
@DataProvider(name = "loginData")
public Object[][] getLoginData() {
    return new Object[][] {
        {"admin",   "admin123",  true,  "Dashboard"},
        {"invalid", "wrong",     false, "Invalid credentials"},
        {"user1",   "pass1",     true,  "Home"},
    };
}

@Test(dataProvider = "loginData")
public void loginTest(String user, String pass, boolean shouldPass, String expected) {
    page.navigate("/login");
    page.getByLabel("Username").fill(user);
    page.getByLabel("Password").fill(pass);
    page.getByRole(AriaRole.BUTTON, new
Page.GetByRoleOptions().setName("Login")).click();
    if (shouldPass) {
        assertThat(page).hasTitle(expected);
    } else {
        assertThat(page.locator(".error")).hasText(expected);
    }
}
```

Excel Data Provider (Apache POI)

```
// utils/ExcelUtils.java
public class ExcelUtils {
    public static Object[][] readExcel(String filePath, String sheetName) throws
Exception {
        Workbook wb = WorkbookFactory.create(new File(filePath));
        Sheet sheet = wb.getSheet(sheetName);
        int rows = sheet.getLastRowNum();
        int cols = sheet.getRow(0).getLastCellNum();
        Object[][] data = new Object[rows][cols];
        for (int i = 1; i <= rows; i++) {
            Row row = sheet.getRow(i);
            for (int j = 0; j < cols; j++) {
                data[i-1][j] = row.getCell(j).toString();
            }
        }
        wb.close();
        return data;
    }
}

@DataProvider(name = "excelData")
public Object[][] getExcelData() throws Exception {
    return ExcelUtils.readExcel("src/test/resources/testdata.xlsx", "LoginData");
}
```

SECTION 13 – CI/CD INTEGRATION

GitHub Actions Workflow

```
# .github/workflows/playwright.yml
name: Playwright Tests
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'

      - name: Install Playwright browsers
        run: mvn exec:java -e -D exec.mainClass=com.microsoft.playwright.CLI -D
exec.args="install --with-deps"

      - name: Run tests
        run: mvn clean test -Dheadless=true -Dbrowser=chromium

      - name: Upload Allure results
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: allure-results
          path: target/allure-results
```

Running Tests from Command Line

```
# Run all tests
mvn clean test

# Run with specific browser
mvn clean test -Dbrowser=firefox

# Run headless
mvn clean test -Dheadless=true

# Run specific test class
mvn clean test -Dtest=LoginTest

# Run specific method
mvn clean test -Dtest=LoginTest#validLoginTest

# Run with TestNG XML
mvn clean test -DsuiteXmlFile=testng.xml

# Generate Allure report after run
mvn allure:serve
```

SECTION 14 – BEST PRACTICES & TIPS

Do's and Don'ts

✅ DO	❌ DON'T
Use getByRole, getByLabel, getByText	Use brittle CSS selectors or XPath
Use POM design pattern	Put all code in test methods
Use assertThat() Playwright assertions	Use Thread.sleep() for waits
Create isolated BrowserContexts per test	Share one Page across tests
Store credentials in config/env vars	Hardcode passwords in tests
Use storageState to reuse login	Login via UI in every test
Record traces for failed tests	Debug by re-running tests manually
Use parallel execution in CI	Run tests sequentially in CI

Common Issues & Fixes

Element not found	Check locator; use page.pause() to debug; ensure page is loaded
Flaky tests	Avoid fixed delays; use waitFor or assertions; check auto-wait conditions
Timeout error	Increase timeout in options; check network speed; use NETWORKIDLE state
StaleElement error	Playwright handles this automatically — no action needed
Tests not parallel	Use @BeforeMethod not @BeforeClass; ensure no shared state
Video not saving	Close context (not browser) to finalize video file
SSL errors	Use setIgnoreHTTPSErrors(true) in context options
Shadow DOM	Use page.locator("host >> shadow >> selector")

Useful Utility Methods

```
// utils/PlaywrightUtils.java

// Take screenshot with timestamp
public static void screenshot(Page page, String name) {
    String ts =
LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyyMMdd_HHmmss"));
    page.screenshot(new Page.ScreenshotOptions()
        .setPath(Paths.get("screenshots/" + name + "_" + ts + ".png"))
        .setFullPage(true));
}
```

```
// Wait for spinner to disappear
public static void waitForLoader(Page page) {
    page.locator(".spinner").waitFor(
        new
Locator.WaitForOptions().setState(WaitForSelectorState.HIDDEN).setTimeout(15000));
}

// Highlight element (for debugging)
public static void highlight(Locator locator) {
    locator.evaluate("el => el.style.border = '3px solid red'");
}

// Get table data as List<Map>
public static List<Map<String,String>> getTableData(Page page, String tableSelector) {
    List<Map<String,String>> data = new ArrayList<>();
    List<String> headers = page.locator(tableSelector + " th")
        .allTextContents();
    List<Locator> rows = page.locator(tableSelector + " tbody tr").all();
    for (Locator row : rows) {
        List<String> cells = row.locator("td").allTextContents();
        Map<String,String> rowMap = new LinkedHashMap<>();
        for (int i = 0; i < headers.size(); i++) rowMap.put(headers.get(i),
cells.get(i));
        data.add(rowMap);
    }
    return data;
}
```

QUICK REFERENCE CHEAT SHEET

Complete Method Reference

page.navigate(url)	Open a URL
page.title()	Get page title
page.url()	Get current URL
page.reload()	Reload page
page.goBack() / goForward()	Browser navigation
page.waitForURL(pattern)	Wait until URL matches
page.waitForLoadState()	Wait for LOAD/DOM/NETWORKIDLE
page.locator(selector)	Find element(s)
locator.click()	Click element
locator.fill(text)	Clear and type text
locator.type(text)	Type char by char
locator.press(key)	Press key on element
locator.hover()	Mouse hover
locator.isVisible()	Returns boolean
locator.isEnabled()	Returns boolean
locator.textContent()	Get visible text
locator.getAttribute(name)	Get attribute value
locator.all()	Returns List<Locator>
locator.count()	Number of matched elements
locator.nth(n)	Get nth element (0-based)
locator.waitFor()	Wait for default visible state
assertThat(page).hasTitle(t)	Assert page title
assertThat(locator).isVisible()	Assert element visible
assertThat(locator).hasText(t)	Assert text content
apiContext.get(path)	HTTP GET request
apiContext.post(path, opts)	HTTP POST request
response.status()	HTTP status code
response.text()	Response body as string
response.ok()	True if 2xx status

context.storageState(opts)	Save auth cookies/storage
page.screenshot()	Take screenshot (bytes)
context.tracing().start()	Start trace recording

Happy Testing with Playwright + Java! 🚀